AtmoVista - Internship Project Report

AQI forecasting for Pakistan with MLOps, live deployment, and GitHub Actions automation.

Project title: AtmoVista (Pearls AQI Predictor)

Forecast window: Next 3 days (+24h, +48h, +72h)

Cities: Lahore, Karachi, Islamabad, Peshawar, Quetta

Repository: GitHub (with GitHub Actions)

Deployment: API on Render, UI on Vercel, features/models on Hopsworks

1. Introduction

For my internship project I built AtmoVista, a system that forecasts Air Quality Index (AQI) for five major cities in Pakistan over the next three days. I collected data, engineered features, trained and compared multiple models, stored everything in Hopsworks, and deployed a working web app that can be opened live.

The main parts of the project are:

- data collection from Open-Meteo
- feature engineering and backfill
- model training (Ridge, Random Forest, HistGradientBoosting, LSTM)
- Hopsworks feature store and model registry
- GitHub Actions for hourly features and daily training
- FastAPI backend + React frontend (AtmoVista dashboard)

I wanted the app to be useful for normal users (current AQI, 3-day outlook, health tips) and also clear enough for technical review (model scores, SHAP, monitoring).

2. Problem statement

Air quality in Pakistan changes a lot between cities and seasons, especially during smog months. Based on the internship brief, I focused on these requirements:

- Forecast AQI for +24h, +48h, and +72h
- Keep data updated on a schedule
- Explain why the model predicts a certain level (SHAP / XAI)
- Show alerts for unhealthy or hazardous air
- Use tools that work on free or low-cost hosting

AtmoVista covers these through a full pipeline and a dashboard with two modes: For you (everyday use) and For experts (models, explainability, accuracy).

3. Objectives completed

Objective	Status
Predict AQI for next 3 days	Done
Feature pipeline (fetch to feature store)	Done
Historical backfill for training	Done
Train and evaluate multiple models	Done
Store models in model registry	Done
Automate hourly and daily pipelines	Done

Objective	Status
Interactive AtmoVista dashboard	Done
EDA	Done
SHAP explainability	Done
Hazardous AQI alerts	Done
AQI Copilot chat (Groq + fallback)	Done
GitHub + Render + Vercel + Hopsworks	Done

The brief suggested Streamlit or Gradio. My mentor allowed FastAPI + React instead, so I built a custom UI for better presentation marks.

4. System overview

This is how the deployed system works:

- Open-Meteo APIs provide pollutant and weather data
- Feature and backfill pipelines engineer hourly features
- Features are stored in the Hopsworks Feature Store
- Training compares Ridge, Random Forest, HistGradientBoosting, and LSTM
- Winning models are registered in the Hopsworks Model Registry
- FastAPI loads models/features and serves forecasts
- The AtmoVista React dashboard consumes the API
- GitHub Actions keeps features and models updated on schedule

Main layers

Layer	Responsibility
Data	Open-Meteo air quality and weather
Features	Time, lag, rolling, future-weather features and targets
Storage	Hopsworks Feature Group and Model Registry
Training	Per-horizon model selection
Serving	FastAPI inference API
Presentation	AtmoVista React dashboard
Automation	GitHub Actions hourly and daily jobs

5. Technology stack

Area	Choice	Why I used it
Language	Python	Required for ML pipelines
Classical ML	Scikit-learn	Ridge / RF / HistGB

Area	Choice	Why I used it
Deep learning	TensorFlow LSTM	Required advanced model
Feature store	Hopsworks	Feature Store + Model Registry in brief
Orchestration	GitHub Actions	Free CI/CD (instead of Airflow)
API	FastAPI	Serve forecasts to the UI
UI	React + Vite + TypeScript	Custom frontend (mentor approved)
Explainability	SHAP	Required XAI; per-horizon signed drivers
Deploy	Render + Vercel	Host API and frontend

6. Data collection

I used the Open-Meteo Air Quality API and Weather API because they are free and do not need a paid key. The brief mentioned AQICN/OpenWeather as examples; Open-Meteo was enough for this project.

Pollutants / AQI

- PM2.5, PM10, CO, NO2, SO2, O3
- US AQI (preferred) / European AQI fallback

Weather

- temperature, humidity, precipitation
- wind speed / direction
- cloud cover, surface pressure

Cities and coordinates

City	Latitude	Longitude
Lahore	31.5204	74.3587
Karachi	24.8607	67.0011
Islamabad	33.6844	73.0479
Peshawar	34.0151	71.5249
Quetta	30.1798	66.9750

Training scale

Item	Value
Backfill window	365 days
Dataset size (EDA)	43,800 hourly rows
Cities covered	5

7. Feature engineering

I implemented features in src/features/engineering.py.

Group	Examples
Raw signals	pollutants, weather, current AQI
Time features	hour, day, day_of_week, month, sin/cos encodings
Derived features	AQI change rate, temp x humidity, wind x PM2.5
Lag features	AQI and PM2.5 lags at 1, 3, 6, 12, 24, 48, 72 hours
Rolling stats	mean/std over 6h, 12h, 24h windows
Future weather	weather at t+24 / t+48 / t+72
Targets	aqi_target_24h, aqi_target_48h, aqi_target_72h

Design choices:

- Separate model per horizon (+24h, +48h, +72h) - better accuracy in my experiments
- Controlled noise on future-weather features during training (realistic mode)
- Prediction intervals (80%) from validation residuals

8. Exploratory data analysis (EDA)

EDA notebook: notebooks/eda.ipynb. Plots include timeseries, correlation heatmap, seasonality, and city boxplots.

What I found:

- Lahore has the highest median AQI and the biggest spikes (smog).
- Karachi is the most stable city with the lowest mean AQI in my dataset.
- PM2.5 and PM10 correlate most strongly with AQI.
- Evening/night and winter months (Nov to Jan) tend to have higher AQI on average.
- Each city behaves differently, so error varies by city.

City AQI summary (from EDA)

City	Mean	Std	Median	Max
Islamabad	113.0	30.7	107.0	206
Karachi	90.1	21.8	85.0	165
Lahore	153.0	49.1	154.0	364
Peshawar	114.6	29.0	111.0	212
Quetta	86.4	31.7	77.0	332

Forecast difficulty by city

Difficulty	Cities
Hardest	Lahore, Quetta
More stable	Karachi, Peshawar

9. Training pipeline

Script: pipelines/training_pipeline.py

Models experimented

Model	Library
Ridge Regression	Scikit-learn
Random Forest	Scikit-learn
HistGradientBoosting	Scikit-learn
LSTM	TensorFlow

Evaluation metrics

Metric	Purpose
RMSE	Main selection metric
MAE	Average absolute error
R2	Explained variance
Category accuracy	AQI band match for user-facing quality

Best model is selected per horizon using validation RMSE.

Horizon	Selected model
+24h	Ridge
+48h	HistGradientBoosting
+72h	Ridge

Winner label: ridge@24h+hist_gradient_boosting@48h+ridge@72h | Trained: 2026-08-11

10. Results

10.1 Validation metrics (winner models)

Horizon	Winner	RMSE	MAE	R2	Category Acc.
+24h	Ridge	21.63	14.33	0.716	72.8%
+48h	HistGB	26.22	18.64	0.583	59.9%
+72h	Ridge	28.47	19.78	0.508	61.2%

10.2 Test metrics (winner models)

Horizon	Winner	RMSE	MAE	R2	Category Acc.
+24h	Ridge	20.63	14.77	0.611	67.8%

Horizon	Winner	RMSE	MAE	R2	Category Acc.
+48h	HistGB	26.63	19.26	0.352	58.1%
+72h	Ridge	28.18	20.90	0.275	54.9%

10.3 My reading of the results

- +24h forecasts are the strongest, which is expected.
- Error increases at +48h and +72h.
- All horizons beat a simple persistence baseline in my evaluation.
- Category accuracy matters because users care about bands, not only the exact number.

10.4 Live monitoring (forecast vs actual)

Source: artifacts/monitoring/summary.json

Scope	MAE	RMSE	R2	Category Acc.	Scored rows
Overall	12.33	19.32	0.744	74.8%	3500
+24h	10.42	15.84	0.830	78.8%	1170
+48h	13.48	20.56	0.709	72.1%	1165
+72h	13.10	21.13	0.691	73.4%	1165

Per-city live MAE

City	MAE	Category Acc.	Scored rows
Karachi	4.68	97.6%	700
Islamabad	13.15	66.7%	700
Peshawar	12.95	70.1%	700
Quetta	14.51	71.1%	700
Lahore	16.36	68.3%	700

Karachi tracks best in live monitoring. Lahore is the hardest of the five cities.

11. Feature store and model registry (Hopsworks)

Item	Value
Feature Group	aqi_features (v3)
Feature contents	Engineered hourly features + targets for 5 cities
Model Registry	aqi_forecast_model
Bundle	Per-horizon winners synced by hopsworks_sync.py

In production (STORAGE_MODE=hopsworks) the API reads features from Hopsworks and caches them in memory. Training jobs can still use local parquet when needed.

12. Inference and API

API module: api/main.py

Endpoint area	What it serves
Forecast	Current AQI + 3-day forecast
History	Recent AQI history
Weather	Temperature, humidity, wind, pollutant driver
Health tips / exercise	Practical guidance
Alerts	Hazardous / unhealthy warnings
Copilot	Floating chat over live city data (Groq + fallback)
Explain	Per-horizon SHAP, waterfall, narrative, city compare
Leaderboard / ops / monitoring	Model and pipeline health

Prediction flow:

- Load latest feature row for selected city
- Fill future-weather fields from Open-Meteo forecast
- Run the horizon-specific winner model
- Apply prediction interval
- Map AQI to EPA-style category + color
- Return JSON to the AtmoVista frontend

13. AtmoVista dashboard (frontend)

Built with React + Vite + TypeScript and deployed on Vercel.

For you (everyday mode)

- Pakistan live map + city rankings
- Current AQI gauge with category bands
- Weather strip, pollutant breakdown, 3-day outlook with calendar dates
- Confidence / peak / improvement insights
- Health tips, alerts, exercise advice, smog season panel, city watchlist
- Floating AQI Copilot (bottom-left chat button)

For experts (technical mode)

- Ops strip (storage mode, model, sync)
- Beat-the-baseline panel and pipeline health board
- Model leaderboard
- XAI: per-horizon SHAP, signed local drivers, waterfall, narrative
- Global drivers, city SHAP compare, glossary hover, pollutant link
- Live accuracy table (forecast vs actual)

14. Alerts, health guidance, and copilot

Feature	Behavior
Alerts	Shown for unhealthy/hazardous current or forecast AQI with date
Health tips	Category-based guidance for masking, exposure, sensitive groups
Exercise advice	Decision card for outdoor exercise based on near-term AQI
AQI Copilot	Chat helper for selected city using live forecast/weather/tips

I added the copilot so people can ask simple questions (exercise, next 24 hours, what to do now) without digging through every panel. It is a floating button on the bottom-left. Backend endpoint: POST /insights/copilot. I used Groq on the free tier (model openai/gpt-oss-20b). The key stays on the Render API only. If Groq is down or the key is missing, the same endpoint falls back to rule-based answers from AtmoVista data.

15. Explainability (SHAP)

Implemented in src/inference/explain.py and shown in the expert dashboard (/aqi/explain, plus compare/global helpers).

View	Purpose
Per-horizon tabs	Separate explanations for +24h, +48h, and +72h winners
City-level importance	Mean |SHAP| over recent city rows
Signed local explanation	Latest-hour SHAP with direction (raises / lowers AQI)
Waterfall	Step path from baseline value to predicted AQI
Plain-language narrative	Short sentence summarizing the top drivers
Global training drivers	Surfaces artifacts/shap_summary.json from training
City compare	Top SHAP drivers across Pakistan cities
Feature glossary	Hover tooltips for lag / weather / pollutant names
Pollutant link	Whether live chemistry and SHAP agree on signal family

Common drivers in my models: recent AQI/PM lags, rolling AQI stats, and weather features.

16. Automation / CI-CD (GitHub Actions)

The repository is on GitHub. Scheduled workflows keep the system updated.

Workflow	Schedule	What it does
feature-hourly.yml	Hourly	Refresh features + reconcile monitoring
training-daily.yml	Daily	Retrain models + sync to Hopsworks
monitor-daily.yml	Daily	Maintain scored forecast-vs-actual history

Repository secrets used by Actions: HOPSWORKS_API_KEY, HOPSWORKS_PROJECT, HOPSWORKS_HOST (optional).

17. Deployment

Service	Platform	Status
AtmoVista frontend	Vercel	Deployed
FastAPI backend	Render (Standard for reliable demo)	Deployed
Feature store / models	Hopsworks	Live source of truth

Production configuration:

- Render uses STORAGE_MODE=hopsworks and Hopsworks credentials
- Vercel uses VITE_API_URL pointing to the Render API
- CORS allows the Vercel frontend origin

18. Extra features (beyond basic brief)

Feature	Reason
5 Pakistan cities	More useful than single-city demo
Two UI modes	Everyday users + technical review
Pollutant breakdown	Context around the AQI number
Signed SHAP + waterfall	Clearer explainability
Confidence bands	Show uncertainty
Smog season + exercise cards	Practical for users
AQI Copilot (Groq + fallback)	Quick Q&A from live city data
Live monitoring	Compare forecasts to actual AQI
Beat-persistence baseline	Show the model adds value

19. Why I chose this methodology

I did not pick tools only because they were in the brief. For each main choice I had a practical reason.

Data source (Open-Meteo)

The brief mentioned AQICN / OpenWeather-style APIs. I chose Open-Meteo because it is free, needs no paid key, and gives both air quality and weather for the same coordinates. That made it easier to join pollutants with temperature, humidity, and wind for features.

Features and horizons

I built lag / rolling / time / future-weather features and trained separate models for +24h, +48h, and +72h. Air quality persistence is strong at short range, so lags help a lot at +24h. Longer horizons need more weather context, and error grows, so picking a winner per horizon was fairer than forcing one model for all three.

Model family (classical + LSTM)

I trained Ridge, Random Forest, HistGradientBoosting, and LSTM. LSTM was required as the advanced option, but I still kept classical models because tabular lag features often work well on hourly AQI. In my runs, Ridge /

HistGB beat LSTM on validation RMSE for the winning horizons. I kept winners by metric, not by which model sounded more advanced.

Hopsworks + GitHub Actions

Hopsworks covers the Feature Store and Model Registry from the brief. For orchestration I used GitHub Actions instead of Airflow: it is free on GitHub, already connected to my repo, and enough for hourly feature refresh plus daily training.

FastAPI + React (not Streamlit / Gradio)

The brief suggested Streamlit or Gradio. After checking with my mentor, I built FastAPI + React so I could design a proper two-mode dashboard (everyday vs experts), map, SHAP views, and a floating copilot. It took more UI work, but the demo looks closer to a real product.

Explainability (SHAP)

I used SHAP because the brief asked for XAI. I went beyond a single bar chart: per-horizon explanations, signed local drivers, waterfall, and a short narrative.

Copilot (Groq + fallback)

I added a floating chat so users can ask simple questions without scrolling every panel. I chose Groq for a free, fast chat API. The key stays on Render only. I also kept a rule-based fallback so the chat still works if the LLM is down or the model name changes.

Render Standard + Vercel

API on Render, UI on Vercel is a common free/low-cost split. I used Render Standard during mentor demos so the API does not sleep mid-review. Vercel hosts the static React build and points VITE_API_URL at Render.

20. Problems I faced and how I fixed them

Problem	What happened / how I fixed it
Empty Open-Meteo JSON	Hourly jobs failed on empty responses; added retries and safer JSON handling
Hopsworks upload 500	Training OK but registry upload failed; added retries in sync + workflow
LSTM not winning	Trained LSTM but classical models won RMSE; selected winners by validation metrics
UI for two audiences	Split dashboard into For you vs For experts
Groq model 404	llama-3.1-8b-instant blocked on free tier; switched to openai/gpt-oss-20b
Copilot without key	Built rule-based fallback from live forecast context
Chat covered header	Capped panel height, scroll inside chat, smoother open/close
Vercel stale build	Frontend lagged after Git push; used production redeploy when needed
Stale updated label	Tied freshness to forecast event_time (latest observation hour)
Hosting limits	Used Render Standard for demos; cached Hopsworks features in the API

These issues were normal for a full MLOps stack. Most fixes were retries, clearer product UX, and choosing free tools that still work under demo pressure.

21. Limitations

- Open-Meteo AQI is model-based, not always the same as a local sensor.
- +48h and +72h errors are naturally higher.
- Hopsworks free usage and hosting costs (Render Standard) need to be watched.
- LSTM was trained but classical models won on most horizons for this data.

22. Future work

- Fuse station-level AQI (where available) for calibration
- Probabilistic category forecasts (not only point AQI)
- City-specific specialist models
- Push notifications for hazardous alerts
- Longer live monitoring window for stronger trust metrics
- Stronger uptime monitoring for the public demo
- Richer copilot (multi-city compare in chat, daily briefing)

23. Conclusion

I completed the internship project as a full AQI forecasting pipeline: data, features, training, Hopsworks, automation, explainability, alerts, and a deployed dashboard with a floating AQI Copilot. The app is on GitHub and runs live on Render and Vercel.

Along the way I had to justify each stack choice and fix real pipeline / deploy / LLM issues. That was as important as the metrics.

Main results:

- Best results at +24h (validation R2 about 0.72, category accuracy about 73%)
- Live monitoring on 3500 scored rows: overall MAE about 12.3, category accuracy about 75%

I think this is good enough for short-term air quality planning in the cities I covered.

24. Appendix - important files

Path	Role
pipelines/feature_pipeline.py	Hourly feature refresh
pipelines/backfill.py	Historical training data
pipelines/training_pipeline.py	Model training and selection
pipelines/hopsworks_sync.py	Feature and model sync
pipelines/monitor_pipeline.py	Forecast vs actual logging
api/main.py	FastAPI serving
src/insights/copilot.py	AQI Copilot (Groq + fallback)
frontend/src/App.tsx	AtmoVista dashboard
frontend/src/components/CopilotPanel.tsx	Floating chat UI
artifacts/model_leaderboard.json	Training results
notebooks/eda.ipynb	EDA

Path	Role
.github/workflows/	CI/CD automation
docs/AtmoVista_Internship_Report.pdf	This report

AtmoVista / Pearls AQI Predictor - internship submission.